Ethics and Legality Integration into the Agile Software Development Pipeline

Social and Ethical Issues

University of Pisa
June 6, 2026

Contents

1 Introduction

Modern software systems increasingly operate as socio-technical infrastructures: they do not merely execute functions, but mediate access to services, process personal data, automate decisions, and shape organisational behaviour. For this reason, a software pipeline cannot be evaluated only in terms of functional correctness, delivery speed, or security. It must also make ethical values and legal obligations explicit, traceable, testable, and monitorable.

This report proposes a progressive engineering model for integrating ethics and legality into a contemporary software development pipeline. The model starts from an Agile sprint loop; to keep the diagrams readable, some feedback arrows are left implicit because iteration is already part of the Agile process.

The Agile loop is expanded through four layers:

1. **Value Sensitive Design (VSD) and Conceptual Engineering (CE)** [2], [3, Secs. 2–4]: values and stakeholders enter the backlog, while ambiguous values are clarified before becoming requirements.
2. **GORE and SLEEC** [4, Secs. 2–3]: value-derived norms are translated into normative goals and then into formal, testable rules.
3. **Compliance-as-Code (CaC)** [5, Secs. 2–4], [6, Secs. 3–5]: ethical and legal requirements are translated into CI/CD gates, policy engines, and audit evidence.
4. **MLOps, Design-time Assurance and Runtime Assurance** [7, Secs. 6.3, 7.1, 8.4.2 and 8.5.2]: AI-specific risks are controlled through model evaluation, monitoring, runtime assurance, and human oversight.

The central thesis is that responsible software engineering requires a shift from *late ethical review* to *continuous ethical and legal assurance*.

2 Conceptual Foundations

2.1 Feedback as the organising principle

The proposed pipeline is based on **feedback loops** that compare observed behaviour with desired goals and then correct the system or the process. Agile retrospectives, CI test failures, policy violations, model drift alerts, incident postmortems, and human override mechanisms all instantiate negative feedback.

This matters because ethical and legal compliance cannot be treated as a static checklist. Values evolve, stakeholders reinterpret impacts, operational environments change, and AI models degrade or behave unexpectedly. Therefore, the pipeline must repeatedly evaluate also if the system still satisfies its goals.

2.2 Risks, Responsibility and the Problem of Many Hands

Modern pipelines distribute action across product owners, developers, data engineers, security engineers, ML engineers, legal experts, platform teams, policy engines, and runtime systems. This creates the classic problem of "many hands": outcomes are collectively produced, while accountability may become diffuse. In AI systems, the problem is amplified because harmful decisions may

depend not only on explicit code, but also on training data, model design, deployment conditions, and runtime adaptation [1].

The proposed model addresses this issue through a **RACI Matrix** placed after the *Sprint Backlog* and before execution. At that point, value-derived norms, GORE/SLEEC rules, technical tasks, CaC policies, and assurance controls have already been selected for the sprint. The RACI Matrix then assigns who is **Responsible** for performing a task, who is **Accountable** for the final decision, who must be **Consulted**, and who must be **Informed**. This supports active responsibility by clarifying who must prevent failures, and passive responsibility by ensuring that decisions, exceptions, deployments, and runtime alerts can later be reconstructed from evidence.

3 Value Sensitive Design and Value-Based Engineering in Agile

3.1 The Agile loop

The initial diagram is a standard Agile loop, of which an example can be found in Figure 1. Its strength is rapid iteration, while its weakness is that values may remain implicit unless they are deliberately introduced into sprint artefacts. For simplicity, the figure shows the main delivery flow; the backward feedback from review and retrospective into *Backlog Refinement* and *Sprint Planning* is implicit in the Agile loop. For this reason, the backlog is not treated only as a queue of features: it is the first place where affected stakeholders, data sources, legal constraints, and foreseeable harms must become visible engineering inputs.

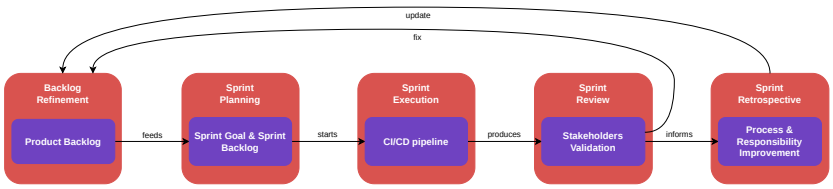

Figure 1: Baseline Agile/Scrum sprint loop, used as the starting point for the progressive integration of ethical and legal assurance.

The papers *Agile as a Vehicle for Values: A Value Sensitive Design Toolkit* and *2025 Trustworthy AI Systems for European Defence* explain how Value Sensitive Design (VSD) and Value-Based Engineering (VBE) can be interpreted and integrated inside the development process by mapping VSD and VBE activities and tools onto ordinary workflow management [2, Secs. 3–5], [7, Sec. 7.1].

3.2 Stakeholder tokens and value source analysis

Within the proposed pipeline, stakeholder tokens and value source analysis are performed inside the *Value Elicitation* activity of *Backlog Refinement*. Stakeholder tokens make affected actors visible during discussion, including direct users, data subjects, maintainers, regulators, vulnerable groups, non-users, and indirectly affected communities. Value source analysis clarifies where values come from: stakeholder expectations, legal duties, organisational principles, professional codes, domain standards, social expectations, and risk assessments.

The output is not yet a technical requirement. It is a set of **value-derived norms** associated with the backlog. These norms constrain user stories and can later be clarified through Conceptual Engineering and refined into GORE/SLEEC requirements.

3.3 Conceptual Engineering

Values such as privacy, autonomy, transparency, and fairness are too abstract to be implemented directly. The Design for Values and Conceptual Engineering literature argues that different conceptualisations of the same value can lead to different design requirements [3, Secs. 2–4]. For this reason, the model places a CE loop inside *Sprint Planning*, before the *Sprint Backlog* is fixed.

The loop has three steps: **Discovery**, which examines how a value is currently understood by stakeholders and in the domain; **Justification**, which asks whether that concept serves the intended moral, legal, and engineering function; and **Construction**, which records the refined concept to be adopted for the system. The output is a value hierarchy that links an abstract value to a norm, then to a design requirement and to a later verification artefact [3, Sec. 5].

This hierarchy also supports legal compliance. For GDPR-relevant systems, values must be traceable to principles such as fairness, transparency, data minimisation, integrity/confidentiality, and accountability, as well as to data protection by design and by default [8, Arts. 5 and 25], [9]. For AI systems, the same hierarchy prepares later controls for the AI Act's risk-based framework and human oversight requirements [10, Art. 14].

3.4 Integration with the Agile model

VSD and CE are integrated into Agile rather than added as an external audit. The Agile-VSD literature shows that values can be introduced into planning, execution, evaluation, and backlog management [2]. Figure 2 represents this by connecting value elicitation, conceptual clarification, value-sensitive implementation, review, and retrospective improvement to the normal sprint loop. The *AI Risks & Data Sources* input extends this logic to data-driven systems, where bias, opacity, privacy risk, or exclusion can arise from the data and modelling choices before any code is deployed.

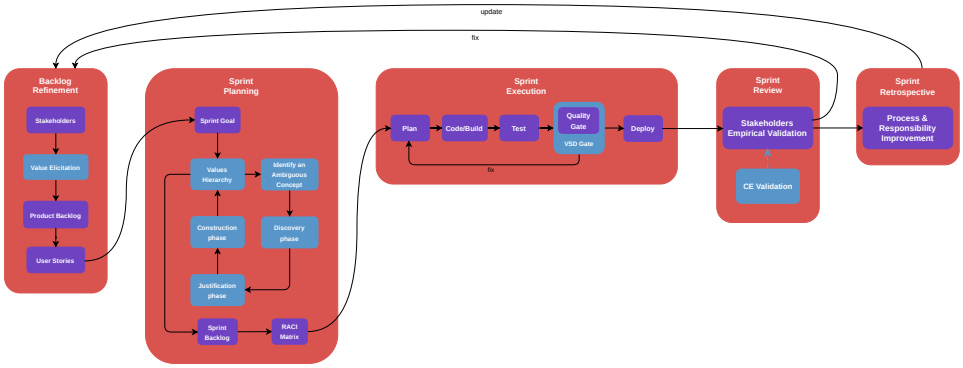

Figure 2: Agile/Scrum sprint loop enriched by Value Sensitive Design, showing how ethical reflection becomes part of ordinary iterative development.

4 Requirements Engineering through GORE and SLEEC

4.1 From values to Norms

The next expansion of the model moves from value analysis to requirements engineering. Goal-Oriented Requirements Engineering (GORE) is used to represent stakeholder intentions as goals. In the ethics-aware adaptation literature, ethical values are represented as **normative goals** aligned

with functional and adaptation goals, enabling analysis of trade-offs and conflicts [4, Sec. 3]. In this stage, values are translated into normative goals, connected to functional or adaptation goals, decomposed into tasks, and finally expressed as rules that can be reviewed and tested. GORE gives the pipeline a place to ask why a requirement exists, not only what the system must do; this matters when competing goals such as privacy, safety, usability, and accountability cannot all be maximised at the same time.

4.2 GORE artefacts and SLEEC rules

A GORE model distinguishes normative goals, functional goals, adaptation goals, obstacles, and trade-offs, which then guide the formulation of SLEEC rules [4, Sec. 3]. This distinction is useful because ethical and legal concerns rarely appear alone: privacy, safety, usability, and accountability often constrain the same feature in different ways.

SLEEC rules translate normative goals into a form that is close to natural language but structured enough for analysis. In the cited work, the DSL background defines rules with the general form "`when <condition> then <response> [within <time>] [unless <exception>]`" [4, Sec. 2]. In the pipeline, *SLEEC Rules* are the bridge between human review and executable control: they make triggers, responses, deadlines, and exceptions precise enough to guide acceptance tests, runtime monitors, and CaC policies.

This stage translates values into technically actionable constraints, but the rules must remain explainable and reviewable. Formal rule execution alone does not guarantee that the underlying value has been interpreted correctly.

4.3 Integration with the Agile-VSD model

Figure 3 shows how the Agile-VSD loop is extended with a requirements branch. Conceptual Engineering clarifies ambiguous values during *Sprint Planning*; *GORE Normative Goals* translate them into structured goals; and *SLEEC Rules* express them as reviewable rules that feed the *Sprint Backlog* and later the CI/CD checks. This makes responsibility more assignable because each value can be traced to a concept, a goal, a rule, and eventually a test or policy.

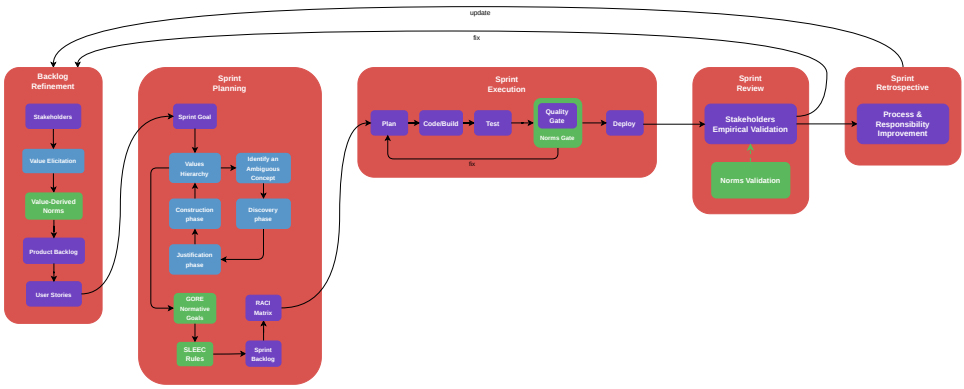

Figure 3: Detailed planning view showing how clarified values are transformed into operational requirements before implementation starts.

5 DevOps and Compliance-as-Code

5.1 CaC integration in CI/CD pipeline

The next expansion moves from requirements specification to the DevOps CI/CD pipeline. Compliance-as-Code (CaC) expresses legal and organisational controls as executable, version-controlled, testable policy artefacts. The CaC literature describes this as a shift from document-centric and retrospective compliance toward programmatic controls embedded directly in the software delivery lifecycle [5, Secs. 2–4].

The design objective is shift-left compliance: checks should run as early as possible, before defects or violations reach production [6]. This is especially important for ethical and legal controls, because they often prevent harm rather than just detect it. For example, a non-compliant release may expose personal data, amplify bias, or make an unexplainable decision; a failed compliance check can block that release before harm occurs.

5.2 Policy-as-Code tools

Policy-as-Code tools such as Open Policy Agent (OPA), HashiCorp Sentinel, Chef InSpec, and Cloud Custodian express compliance constraints in executable form. In the model, these tools populate the *CaC Policy Library*: a governed repository where retention, encryption, access-control, or SLEEC-derived rules can be reviewed, versioned, tested, and connected to audit evidence.

5.3 Integration with the Agile Normative-based model

The previous requirements branch is connected to CI/CD gates. *GORE Normative Goals* and *SLEEC Rules* are translated into **policy-as-code checks**; those checks block or approve pipeline stages, and their decisions are stored as audit evidence. The *CaC Policy Library* in the diagram is the versioned memory of these rules: it prevents compliance from depending on informal recollection and makes policy changes reviewable like source code. The *Quality / CaC Assurance Gate* is the point where the pipeline acts, because a failed legal, security, or ethical control can stop a release before harm reaches production. The *Audit Evidence Store* gives the same process a backward-looking function, preserving policy decisions, exceptions, and deployment records.

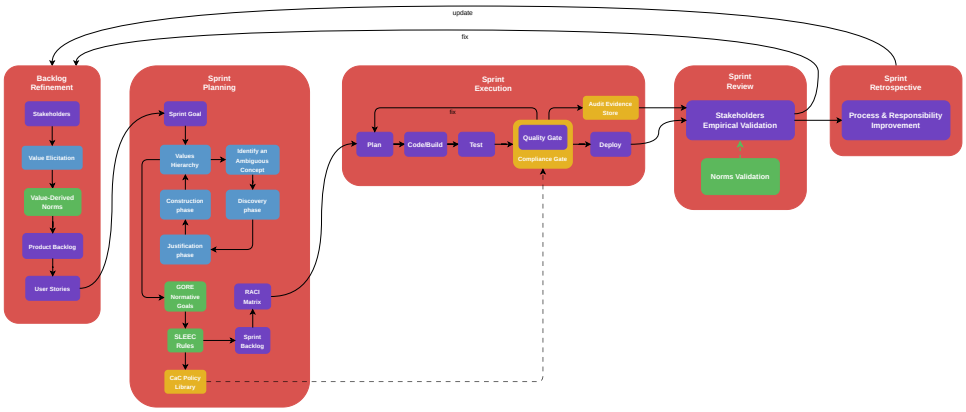

Figure 4: CI/CD expansion showing how compliance checks and audit evidence are integrated into the delivery process.

6 MLOps and Runtime Assurance

6.1 Why AI requires a further expansion

AI systems add **new sources of risk**: training data may be biased, model behaviour may be statistically uncertain, the operational environment may diverge from the training environment, and performance may degrade after deployment. For this reason, responsible AI must be operationalised across governance, process, and system perspectives, not only at the algorithmic level. This requires continuous monitoring, co-versioning of data/model/code/configuration, audit trails, and dynamic ethical risk assessment during operation.

The importance of **operational context** is also visible in the distinction between civilian and military Operational Design Domains (ODDs) discussed in the TAID white paper [7, Secs. 8.4.2]. In civilian applications, the operational domain is often defined around relatively stable and static assumptions, such as expected legal constraints and environmental conditions. In military and adversarial contexts, instead, the operational domain may change rapidly, involve hostile actors, and include uncertainty, degraded information, or mission-specific constraints. This shows why AI assurance cannot depend only on pre-deployment model accuracy: the validity of an AI component also depends on its data sources, risks, stakeholders, and operational conditions.

The ***AI Risks & Data Sources*** block in the diagram expresses this shift. Responsible AI begins before model training, when the team identifies legitimate data sources, potentially under-represented groups, and risks, that must condition the *Value Elicitation* block.

Especially for Autonomous Systems, the DevOps pipeline must also manage data, training, validation, deployment, runtime monitoring, and possible retraining or rollback. The next expansion therefore introduces mechanisms to assure that the AI component is reliable, compliant, and under meaningful human control both before and after deployment.

6.2 Design-time assurance

Design-time assurance takes place before deployment. It checks whether the AI component is reliable, compliant, and fit for its declared operational context [7, Secs. 8.4.2]. This includes data quality and lineage, bias and representativeness, model validation, robustness and generalisation, explainability, and conformity with legal or organisational policies.

The key added control is the *Model Evaluation Gate*: a trained model is not accepted merely because it runs, but because evidence supports deployment under known requirements and residual risks. This mainly supports active responsibility, because foreseeable harms must be assessed and mitigated before release.

6.3 Runtime assurance

Runtime assurance takes place after deployment and checks whether the AI-enabled system remains trustworthy under real operating conditions. It includes runtime monitoring, drift detection, operational-domain checks, fallback or rollback mechanisms, escalation, and evidence collection. In this sense, runtime assurance is broader than monitoring: it observes deviations and provides mechanisms to respond to them [7, Secs. 8.4.2.2 and 9.2.3].

It supports active responsibility by preventing uncontrolled degradation, and passive responsibility by preserving evidence for incident reconstruction.

6.4 Meaningful Human Control

Meaningful Human Control (MHC) prevents technological autonomy from eliminating human accountability. So, one of the goals of the model is to ensure meaningful human control within runtime assurance: monitors detect uncertainty, drift, operational-domain violations, or high-impact decisions, and escalation thresholds route those cases to a human operator. MHC should therefore provide context and explanations, real authority to override or stop the system, logging of human and machine decisions, and clear role definitions for operators [7, Secs. 6.3, 7.1 and 8.5.2].

6.5 Agile Normative & Risk-prevention based model

In Figure 5, the *AI Risks & Data Sources* inputs enter *Backlog Refinement* before *Sprint Planning*, so the system starts from the people, data, and harms that may be affected. *Sprint Planning* then contains the CE loop, where ambiguous values are clarified before they are accepted as engineering commitments. *GORE Normative Goals* and *SLEEC Rules* translate those commitments into goals and rules, the *CaC Policy Library* stores the executable policy version of those rules, and the *Quality / CaC Assurance Gate* checks them during delivery. The *Audit Evidence Store* preserves the decisions produced by these gates, while the *Model Evaluation Gate* and *Runtime Assurance* blocks keep the AI component under control before and after deployment.

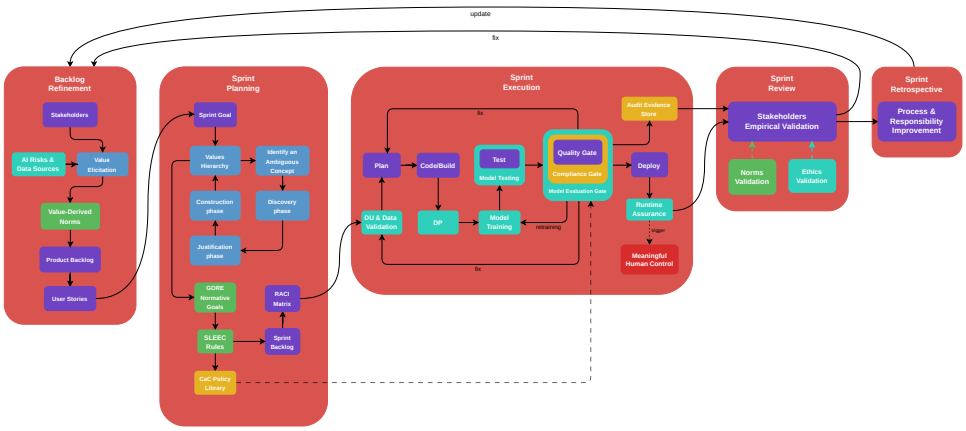

Figure 5: Final pipeline with the full progression from value elicitation to design-time and runtime assurance related to the MLOps integration.

7 Conclusions

The model proposed in this report shows **how ethical and legal concerns can be moved from external review into the normal engineering pipeline**. Agile provides the iterative structure; VSD/VBE and Conceptual Engineering make stakeholders, values, and ambiguous concepts explicit; GORE/SLEEC translate those values into normative goals and reviewable rules; CaC turns selected rules into executable checks and audit evidence; and MLOps adds design-time and runtime assurance for AI systems.

From a **legal perspective**, the strength of the model is **traceability**. GDPR principles such as data minimisation, transparency, accountability, and data protection by design can be connected to backlog items, norms, SLEEC rules, CaC policies, and audit evidence. The same logic supports AI Act-style risk management: AI risks and data sources enter the backlog early, while model evaluation, runtime monitoring, and human oversight become part of the assurance process.

From a **risk perspective**, the model is **preventive** rather than merely reactive. The CaC gate can block non-compliant releases, while runtime assurance can detect drift, abnormal behaviour, or violations of the approved operational context after deployment. However, the model does not remove risk completely. It depends on the quality of value elicitation, the correctness of rule translation, the coverage of tests and policies, and the ability of humans to interpret runtime evidence.

Human-in-the-loop control is therefore not treated as a symbolic approval step. Meaningful Human Control is useful only if operators receive understandable explanations, have authority to intervene, and are supported by logs, escalation paths, and clear responsibility assignments. This is also where the RACI matrix is important: it reduces the responsibility gap by linking tasks, rules, gates, exceptions, and runtime decisions to identifiable roles.

Practically, the model is feasible because it extends artefacts already familiar to software teams: backlogs, sprint planning, CI/CD gates, policy repositories, model evaluation, monitoring, and retrospectives. Its main strength is that it creates a **continuous chain from values to evidence**. Its main limitation is complexity: not every ethical or legal concern can be fully automated, and overloading the pipeline with too many gates may slow development or create false confidence. The model should therefore be used as a disciplined framework for making responsibility, compliance, and AI risk explicit, not as a guarantee that ethical behaviour can be completely reduced to code.

A MLOps Assurance Diagram

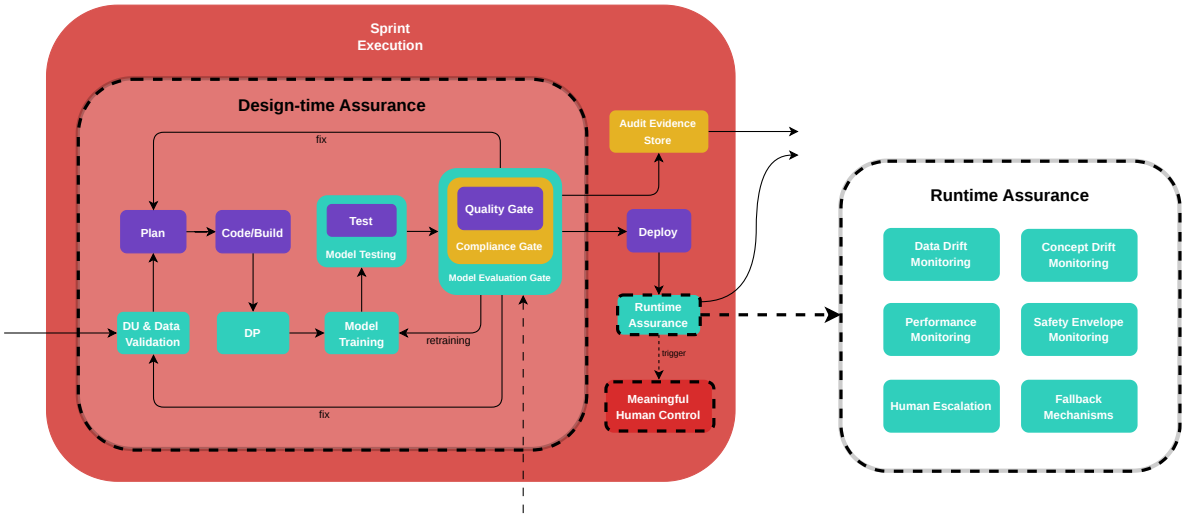

Figure 6: MLOps assurance view, distinguishing the checks performed before deployment from the monitoring and control mechanisms active after release.

B Value Hierarchy Examples

Value	Norm	Requirement	Verification
Privacy	Data minimisation	Collect only fields required for the declared purpose.	Schema test; privacy review
Autonomy	Meaningful user control	Provide users with the ability to change their choices or appeal decisions.	UX test; audit log
Fairness	Non-discrimination	Evaluate model performance across relevant groups.	Fairness test report
Transparency	Explainability	Provide understandable reasons for consequential decisions.	Explanation acceptance test
Security	Integrity and confidentiality	Encrypt personal data at rest and in transit.	IaC/policy test

Table 1: Illustrative examples of value hierarchy entries.

C SLEEC Rule Examples

Listing 1: Example SLEEC-style rule for GDPR deletion.

```
rule delete_personal_data:
  when UserRequestsErasure and requestIsValid
  then DeletePersonalData within 30 days
  unless legalRetentionObligation
```

```
  then RestrictProcessing and NotifyUser within 7 days
```

Listing 2: Example SLEEC-style rule for human oversight.

```
rule human_review_high_risk_decision:
  when HighRiskAIDecisionGenerated and confidence < threshold
  then EscalateToHumanReviewer within 5 minutes
```

D Turing and Searle as Warnings for Responsible AI

The Turing-test tradition frames intelligence behaviourally: a machine may appear intelligent if its outputs are indistinguishable from those of a human. However, software engineering for responsible AI cannot rely only on output imitation. A system may produce convincing behaviour while still violating privacy, amplifying bias, or hiding causal responsibility. The distinction between weak and strong AI is therefore operationally relevant: most deployed AI systems should be engineered as powerful decision-support artefacts, not as autonomous moral subjects.

Also the Searle's Chinese Room argument is useful as a warning for formal compliance. A rule engine may manipulate symbols correctly while lacking semantic understanding of the underlying value. This is not an argument against formalisation; rather, it shows why formal rules must remain connected to stakeholder interpretation, domain expertise, legal review, and human oversight. In the proposed pipeline, SLEEC rules and policy-as-code are therefore treated as *engineering artefacts requiring governance*, not as replacements for judgement.

References

[1] A. Matthias. *The responsibility gap: Ascribing responsibility for the actions of learning automata*. Vol. 6, pp. 175–183, 2004. DOI: https://doi.org/10.1007/s10676-004-3422-1.

[2] S. Umbrello and O. Gambelin. *Agile as a Vehicle for Values: A Value Sensitive Design Toolkit*. In *Rethinking Technology and Engineering: Dialogues Across Disciplines and Geographies*, 2023. Available at: https://iris.unito.it/handle/2318/1885145.

[3] H. Veluwenkamp and J. van den Hoven. *Design for values and conceptual engineering*. Vol. 25, article 2, 2023. DOI: https://doi.org/10.1007/s10676-022-09675-6.

[4] E. Silva Júnior, L. Marsso, R. Caldas, M. Chechik, and G. N. Rodrigues. *Operationalizing Human Values in the Requirements Engineering Process of Ethics-Aware Autonomous Systems*. arXiv:2602.09921, 2026. Available at: https://arxiv.org/abs/2602.09921.

[5] L. Potharalanka. *Compliance-As-Code: A Framework for Regulatory Automation in Enterprise CI/CD Pipelines*. Sarcouncil Journal of Engineering and Computer Sciences, vol. 4, issue 7, pp. 257–267, 2025. Available at: https://www.sarcouncil.com/download-article/SJECS-145_-2025-257-267.pdf.

[6] P. Singh. *Integrating Compliance-as-Code into CI/CD Pipelines for Regulated Industries*. IJIRCT, vol. 11, issue 2, 2025. Available at: https://www.ijirct.org/viewPaper.php?paperId=2506009.

[7] European Defence Agency. *Trustworthiness for AI in Defence: Developing Responsible, Ethical, and Trustworthy AI Systems for European Defence*. TAID White Paper, version 1.0, 09/05/2025. Available at: https://eda.europa.eu/docs/default-source/brochures/taid-white-paper-final-09052025.pdf.

[8] European Parliament and Council of the European Union. *Regulation (EU) 2016/679: General Data Protection Regulation*. 2016. Available at: https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng. See especially Articles 5 and 25.

[9] European Data Protection Board. *Guidelines 4/2019 on Article 25 Data Protection by Design and by Default*. Version 2.0, adopted on 20 October 2020. Available at: https://www.edpb.europa.eu/our-work-tools/our-documents/guidelines/guidelines-42019-article-25-data-protection-design-and_en.

[10] European Parliament and Council of the European Union. *Regulation (EU) 2024/1689: Artificial Intelligence Act*. 2024. Available at: https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng. See especially Article 14 on human oversight.